

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ
по лабораторной работе №3
по дисциплине «Построение и Анализ Алгоритмов»
Тема: Вычисление редакционного расстояния и предписания алгоритмом
Вагнера-Фишера.

Студент гр. 4342

Ларионов В.А.

Преподаватель

Уткина О.Н.

Санкт-Петербург

2026

Задание.

Над строкой ε (будем считать строкой непрерывную последовательность из латинских букв) заданы следующие операции:

1. $\text{replace}(\varepsilon, a, b)$ – заменить символ a на символ b .
2. $\text{insert}(\varepsilon, a)$ – вставить в строку символ a (на любую позицию).
3. $\text{delete}(\varepsilon, b)$ – удалить из строки символ b .

Каждая операция может иметь некоторую цену выполнения (положительное число).

Даны две строки A и B , а также три числа, отвечающие за цену каждой операции. Определите минимальную стоимость операций, которые необходимы для превращения строки A в строку B .

Входные данные: первая строка – три числа: цена операции replace , цена операции insert , цена операции delete ; вторая строка – A ; третья строка – B .

Выходные данные: одно число – минимальная стоимость операций.

Даны две строки A и B , а также три числа, отвечающие за цену каждой операции. Определите последовательность операций (редакционное предписание) с минимальной стоимостью, которые необходимы для превращения строки A в строку B .

Пример (все операции стоят одинаково)

M	M	M	R	I	M	R	R
C	O	N	N		E	C	T
C	O	N	E	H	E	A	D

Пример (цена замены 3, остальные операции по 1)

М	М	М	Д	М	І	І	І	І	Д	Д
С	О	Н	Н	Е					С	Т
С	О	Н		Е	Н	Е	А	Д		

Выходные данные: первая строка – последовательность операций (М – совпадение, ничего делать не надо; R – заменить символ на другой; І – вставить символ на текущую позицию; D – удалить символ из строки); вторая строка – исходная строка A; третья строка – исходная строка B.

Вар. 2. "Особый заменитель и особо удаляемый символ": цена замены на определённый символ отличается от обычной цены замены; цена удаления другого (или того же) определённого символа отличается от обычной цены удаления. Особый заменитель и цена замены на него, особо удаляемый символ и цена его удаления — дополнительные входные данные.

Описание алгоритма.

Задача состоит в вычислении редакционного расстояния между двумя строками с возможностью назначения индивидуальных стоимостей для операций с определенными символами. Пользователь задаёт две строки, специальный заменяемый символ с ценой его замены и специальный удаляемый символ с ценой его удаления. Базовые стоимости операций (замена, удаление, вставка) задаются макросами *REPCOST*, *DELCOST* и *INSCOST* (по умолчанию 1).

Для решения применяется классический алгоритм динамического программирования Вагнера-Фишера. Его основная идея -- итеративное заполнение матрицы расстояний, где ячейка (i, j) содержит минимальную стоимость преобразования первых i символов первой строки в первые j символов второй строки. На каждом шаге для пары символов вычисляются стоимости трех операций: замена (с учетом того, является ли символ второй строки специальным заменяемым), удаление (с учетом того, является ли символ первой строки специальным удаляемым) и вставка (всегда с базовой

стоимостью). Выбирается операция с минимальной стоимостью; при равенстве приоритет отдаётся замене, затем удалению, затем вставке.

Параллельно с вычислением расстояний заполняется матрица операций *choice*, где для каждой ячейки сохраняется символ выполненной операции (*M* -- совпадение, *R* -- замена, *D* -- удаление, *I* -- вставка). После заполнения всей матрицы выполняется обратный ход: начиная с правого нижнего угла, по матрице *choice* восстанавливается последовательность операций (редакционное предписание), которая затем разворачивается и выводится вместе с итоговым расстоянием.

Оптимизации алгоритма.

1. Использование двух векторов вместо полной матрицы расстояний. Вместо хранения всей матрицы размером $(N+1)$ на $(M+1)$ алгоритм хранит только предыдущую (*prevDist*) и текущую (*currDist*) строки расстояний. Это снижает затраты памяти с $O(N \cdot M)$ до $O(M)$.

2. После обработки каждой строки векторы меняются местами с помощью эффективного *std::swap*, не требующего копирования данных.

3. Вычисление специальных стоимостей вынесено в *inline*-функции. Функции *getDelCost* и *getRepCost* объявлены как *inline*, что исключает накладные расходы на вызов и позволяет компилятору встроить проверку на специальный символ непосредственно в место вызова, ускоряя выполнение внутреннего цикла.

4. Однократное вычисление стоимости удаления. Для каждого символа первой строки стоимость его удаления вычисляется один раз перед входом во внутренний цикл, а не пересчитывается для каждой ячейки строки. Это снижает количество операций сравнения с *M* до *1* на строку матрицы.

5. Немедленное ветвление при совпадении символов. Если текущие символы строк совпадают, стоимость совпадения равна 0 (*distance* не увеличивается), и выбор других операций не рассматривается, что экономит три вычисления стоимости и несколько сравнений для каждой совпавшей пары символов.

Сложность алгоритма.

Временная сложность: $O(N \cdot M)$, где N и M -- длины первой и второй строк соответственно. Для каждой из N строк выполняется проход по всем M столбцам с вычислением константного числа операций. Сложность подтверждена замерами времени выполнения на Рисунке 1.

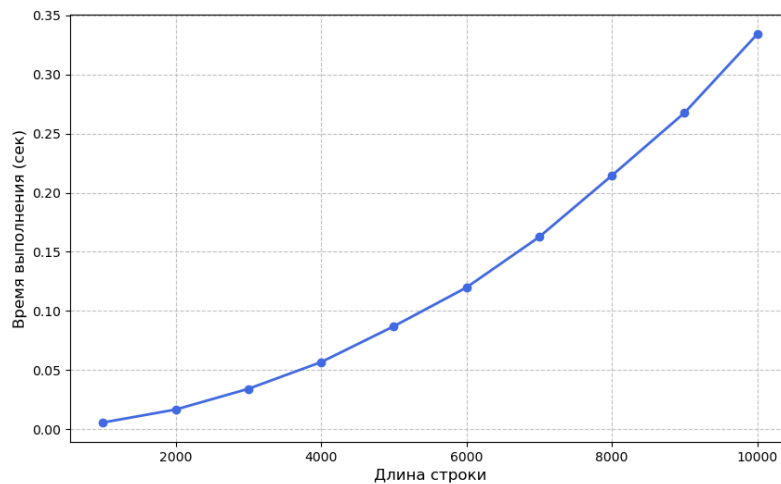


Рисунок 1 - график зависимости
времени выполнения от длины строки

Пространственная сложность: $O(M)$ для хранения двух векторов расстояний и $O(N \cdot M)$ для матрицы операций *choice*, которая необходима для восстановления редакционного предписания. Без необходимости восстановления предписания сложность по памяти может быть снижена до $O(M)$ путём отказа от хранения матрицы *choice*.

Описание функций и структур данных.

struct infoStruct - хранит исходные данные задачи: две строки (*firstString*, *secondString*), специальные символы замены и удаления (*replaceChar*, *deleteChar*) и стоимости операций с ними (*replaceTargetCharCost*, *deleteTargetCharCost*).

getInput() - считывает из стандартного ввода две строки, затем специальный заменяемый символ с ценой его замены, затем специальный удаляемый символ с ценой его удаления. Возвращает *false* при ошибке ввода.

getDelCost() - возвращает стоимость удаления символа: *specialDeleteCost*, если символ совпадает со специальным удаляемым, иначе *DELCOST* (по умолчанию 1).

getRepCost() - возвращает стоимость замены символа: *specialReplaceCost*, если символ совпадает со специальным заменяемым, иначе *REPCOST* (по умолчанию 1).

printResult() - восстанавливает редакционное предписание обратным ходом по матрице операций *choice*, выводит в консоль редакционное расстояние и предписание. При определённом макросе *LOGGING* дублирует вывод в лог-файл.

matrixToString() - (только при *LOGGING*) формирует строковое представление матрицы операций для журналирования.

main() - основная функция: получает входные данные через *getInput()*, инициализирует два вектора расстояний (предыдущую и текущую строки) и матрицу операций *choice*. Итеративно заполняет расстояния по алгоритму Вагнера-Фишера, учитывая специальные стоимости замены и удаления, выбирая минимальную из операций замены, удаления и вставки (при равных *cost* приоритет: замена > удаление > вставка). После заполнения вызывает *printResult()* и завершает работу.

Тестирование.

Тестирование приведено в таблице 1 в Приложении Б.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл src/algo.cpp

```
#include <algorithm>
#include <iostream>
#include <fstream>
#include <vector>
#include <string>

using namespace std;

#ifndef REPCOST
#define REPCOST 1
#endif
#ifndef DELCOST
#define DELCOST 1
#endif
#ifndef INSCOST
#define INSCOST 1
#endif

#ifdef LOGGING
ofstream logFile("file.log");
#endif

/*
Структура для хранения и передачи информации о вводе.
    firstString - первая строка
    secondString - вторая строка
    replaceChar - специальный заменяемый символ
    deleteChar - специальный удаляемый символ
    replaceTargetCharCost - стоимость замены на спец. заменяемый символ
    deleteTargetCharCost - стоимость удаления спец. удаляемого символа
*/
struct infoStruct {
    string firstString = "";
    string secondString = "";
    char replaceChar = 'a';
    char deleteChar = 'a';
    int replaceTargetCharCost = 1;
    int deleteTargetCharCost = 1;
};

/*
Функция для получения ввода. Ожидает ввод в формате
<Строка_1>
<Строка_2>
<Спец. заменяемый символ> <цена его замены> <Спец. удаляемый символ>
<цена его замены>
Возвращает true, если ввод принят корректно и false, если что-то пошло
не так
*/
bool getInput(infoStruct& input) {
```

```

    if (!getline(cin, input.firstString)) {
        return false;
    }
    if (!getline(cin, input.secondString)) {
        return false;
    }
    if (!(cin >> input.replaceChar >> input.replaceTargetCharCost)) {
        return false;
    }
    if (!(cin >> input.deleteChar >> input.deleteTargetCharCost)) {
        return false;
    }
    return true;
}

/*
Функция для получения корректной стоимости удаления текущего символа.
Проверяет является ли char currentChar специальным удаляемым символом
specialDeleteChar, если да, то возвращает int specialDeleteCost,
в противном случае DELCOST (по умолчанию 1)
*/
inline int getDelCost(
    char currentChar,
    char specialDeleteChar,
    int specialDeleteCost
) {
    return (currentChar == specialDeleteChar) ? specialDeleteCost :
DELCOST;
}

/*
Функция для получения корректной стоимости замены текущего символа.
Проверяет является ли char currentChar специальным заменяемым символом
specialReplaceChar, если да, то возвращает int specialReplaceCost,
в противном случае REPCOST (по умолчанию 1)
*/
inline int getRepCost(
    char currentChar,
    char specialReplaceChar,
    int specialReplaceCost
) {
    return (currentChar == specialReplaceChar) ? specialReplaceCost :
REPCOST;
}

/*
Функций для вывода результата в формате:
<Редакционное расстояние>
<Редакционное предписание>
На вход принимает матрицу операций vector<vector<char>>>& choice
и значение расстояния int& resultDistance
Ничего не возвращает
*/
inline void printResult(

```



```

        vector<vector<char>>& choice,
        int& resultDistance
    ) {
        // Восстанавливаем путь обратным ходом
        string path;
        size_t i = choice.size() - 1, j = choice[0].size() - 1;
        while (i > 0 || j > 0) {
            char& op = choice[i][j];
            path += op;
            if (op == 'M' || op == 'R') {
                --i; --j;
            } else if (op == 'D') {
                --i;
            } else {
                --j;
            }
        }
        // Путь был собран с конца, переворачиваем
        reverse(path.begin(), path.end());
        // Вывод результата
        cout << resultDistance << endl;
        cout << path << endl;
#ifdef LOGGING
        logFile << "Редакционное расстояние: " << resultDistance << "\n";
        logFile << "Редакционное предписание: " << path << "\n";
#endif
    }

#ifdef LOGGING
    /*
    Функция для журналирования. Принимает матрицу const
    vector<vector<char>>& choice
    возвращает ее в строковом виде
    */
    string matrixToString(const vector<vector<char>>& choice) {
        string strMatrix;
        int n = (int)choice.size();
        if (n == 0) return "";
        int m = (int)choice[0].size();
        for (int i = 0; i < n; ++i) {
            for (int j = 0; j < m; ++j) {
                strMatrix += choice[i][j];
                if (j != m - 1) strMatrix += ' ';
            }
            strMatrix += '\n';
        }
        return strMatrix;
    }
#endif

    /*
    Основной алгоритм, функция создает структуру для вводимых данных,
    ждет ввода через функцию getInput(infoStruct& input), создает два
    сменяющих

```

друг друга вектора, в которые итеративно будут записаны значения расстояния, параллельно с этим в матрицу choice записываются все действия совершенные над строками для восстановления операций и вывода результата через функцию `printResult(vector<vector<char>>& choice, int& resultDistance)`

```

*/
int main() {
    infoStruct input;
    if (!getInput(input)) {
        return EXIT_FAILURE;
    }
    int N = input.firstString.length();
    int M = input.secondString.length();

#ifdef LOGGING
    logFile << "Первая строка: \"" << input.firstString << "\"\n";
    logFile << "Вторая строка: \"" << input.secondString << "\"\n";
    logFile << "Спец. символ замены: '" << input.replaceChar
        << "' (стоимость замены на него: " <<
input.replaceTargetCharCost << ")\n";
    logFile << "Спец. символ удаления: '" << input.deleteChar
        << "' (стоимость удаления: " << input.deleteTargetCharCost <<
")\n";
    logFile << "Базовые стоимости: замена=" << REPCOST
        << ", удаление=" << DELCOST
        << ", вставка=" << INSCOST << "\n\n";
#endif

    vector<int> prevDist(M + 1), currDist(M + 1);
    vector<vector<char>> choice(N + 1, vector<char>(M + 1, '?'));

    // Инициализация первой строки
    for (int j = 0; j <= M; ++j) {
        prevDist[j] = j;
        if (j > 0) {
            choice[0][j] = 'I';
        }
    }

    for (int i = 1; i <= N; ++i) {
        char& currFirstStrChar = input.firstString[i-1];
#ifdef LOGGING
        logFile << "Рассматриваем символ из первый строки: " <<
currFirstStrChar << '\n';
        logFile << "Текущая матрица операций: \n";
        logFile << matrixToString(choice);
        logFile << "\n\n";
#endif
        int delCost = getDelCost(
            currFirstStrChar,
            input.deleteChar,
            input.deleteTargetCharCost
        );
        currDist[0] = prevDist[0] + delCost;
        choice[i][0] = 'D';
    }
}

```

```

        for (int j = 1; j <= M; ++j) {
            char& currSecondStrChar = input.secondString[j-1];
#ifdef LOGGING
            logFile << "Сравниваем с символом из второй строки: " <<
currSecondStrChar << '\n';
#endif
            if (currFirstStrChar == currSecondStrChar) {
                // Совпадение
                currDist[j] = prevDist[j-1];
                choice[i][j] = 'M';
#ifdef LOGGING
                logFile << "Совпадение, расстояние не изменится\n";
#endif
            } else {
#ifdef LOGGING
                logFile << "Символы '" << currFirstStrChar << "' и '"
<< currSecondStrChar << "' не совпали, рассчитываем стоимость
операций\n";
#endif
                // Замена
                int repCost = getRepCost(
                    currSecondStrChar,
                    input.replaceChar,
                    input.replaceTargetCharCost
                );
                int rep = prevDist[j-1] + repCost;
#ifdef LOGGING
                logFile << "Стоимость замены " << rep << '\n';
#endif
                // Удаление
                int del = prevDist[j] + delCost;
#ifdef LOGGING
                logFile << "Стоимость удаления " << del << '\n';
#endif
                // Вставка
                int ins = currDist[j-1] + INSCOST;
#ifdef LOGGING
                logFile << "Стоимость вставки " << ins << '\n';
                logFile << "Выбираем наименьшее. Приоритет R < D <
I\n";
#endif
                if (rep <= del && rep <= ins) {
                    currDist[j] = rep;
                    choice[i][j] = 'R';
#ifdef LOGGING
                    logFile << "Выбрали замену\n\n";
#endif
                } else if (del <= ins) {
                    currDist[j] = del;
                    choice[i][j] = 'D';
#ifdef LOGGING
                    logFile << "Выбрали удаление\n\n";
#endif
                } else {
                    currDist[j] = ins;
                    choice[i][j] = 'I';
#ifdef LOGGING

```

```

                                logFile << "Выбрали вставку\n\n";
#endifif
                                }
                                }
                                }
                                // Переставляем векторы расстояний для следующей итерации
                                prevDist.swap(currDist);
                                }
                                printResult(choice,prevDist[M]);
                                return EXIT_SUCCESS;
                                }

```

Файл src/benchmark.py

```

import subprocess
import time
import random
import string
import matplotlib.pyplot as plt

EXECUTABLE = "./algo"
SIZES = [1000, 2000, 3000, 4000, 5000, 6000, 7000, 8000, 9000, 10000]
ITERATIONS = 3

def generate_data(size):
    chars = string.ascii_letters + string.digits
    line1 = ''.join(random.choice(chars) for _ in range(size))
    line2 = ''.join(random.choice(chars) for _ in range(size))
    return f"{line1}\n{line2}\n1 1 1 1\n"

def run_benchmark():
    results = []
    for size in SIZES:
        total_time = 0
        data = generate_data(size)
        for _ in range(ITERATIONS):
            start = time.perf_counter()
            process = subprocess.Popen(
                [EXECUTABLE],
                stdin=subprocess.PIPE,
                stdout=subprocess.PIPE,
                stderr=subprocess.PIPE,
                text=True
            )
            process.communicate(input=data)
            end = time.perf_counter()
            total_time += (end - start)
        avg_time = total_time / ITERATIONS
        results.append(avg_time)
        print(f"{size:10d} | {avg_time:12.5f}")
    return results

def plot_results(sizes, times):
    plt.figure(figsize=(10, 6))
    plt.plot(sizes, times, marker='o', linestyle='-',
             color='royalblue', linewidth=2)
    plt.xlabel('Длина строки', fontsize=12)
    plt.ylabel('Время выполнения (сек)', fontsize=12)

```

```

plt.grid(True, linestyle='--', alpha=0.7)
plt.savefig('bench_result.png')

if __name__ == "__main__":
    try:
        execution_times = run_benchmark()
        plot_results(SIZES, execution_times)
    except FileNotFoundError:
        print(f"Ошибка: Файл {EXECUTABLE} не найден. Сначала
скомпилируйте его через make.")

```

Файл src/generate_two_string.py

```

import random
import string
import sys

def generate_strings(length=20):
    chars = string.ascii_letters
    line1 = ''.join(random.choice(chars) for _ in range(length))
    line2 = ''.join(random.choice(chars) for _ in range(length))
    print(line1)
    print(line2)
    print("1 1 1 1")

if __name__ == "__main__":
    size = int(sys.argv[1]) if len(sys.argv) > 1 else 20
    generate_strings(size)

```

Файл tests/Makefile

```

alglog: ../src/algo
    @g++ ../src/algo.cpp -O3 -o ../src/algo

alg: ../src/algo
    @g++ ../src/algo.cpp -O3 -DLOGGING -o ../src/algo

tests: ../src/algo
    @echo "Запуск тестов"
    @echo
    @make --no-print-directory t1
    @make --no-print-directory t2
    @make --no-print-directory t3
    @make --no-print-directory t4
    @make --no-print-directory t5

t1: alg
    @echo "Тест 1: одна замена"
    @echo "Ввод:"
    @cat test1.txt

```

```

    @echo "Вывод:"
    @../src/algo < test1.txt
    @echo

t2: alg
    @echo "Тест 2: одно удаление"
    @echo "Ввод:"
    @cat test2.txt
    @echo "Вывод:"
    @../src/algo < test2.txt
    @echo

t3: alg
    @echo "Тест 3: одна вставка"
    @echo "Ввод:"
    @cat test3.txt
    @echo "Вывод:"
    @../src/algo < test3.txt
    @echo

t4: alg
    @echo "Тест 4: одинаковые строки"
    @echo "Ввод:"
    @cat test4.txt
    @echo "Вывод:"
    @../src/algo < test4.txt
    @echo

t5: alg
    @echo "Тест 5: случайные строки разного размера"
    @echo "Ввод: длина строки 1"
    @python3 ../src/generate_two_string.py 1 > test_random.txt
    @echo "a 1 a 1" >> test_random.txt
    @cat test_random.txt
    @echo "Вывод:"
    @../src/algo < test_random.txt
    @echo "Ввод: длина строки 5"
    @python3 ../src/generate_two_string.py 5 > test_random.txt
    @echo "a 1 a 1" >> test_random.txt
    @cat test_random.txt
    @echo "Вывод:"
    @../src/algo < test_random.txt

```

```
@echo "Ввод: длина строки 20"
@python3 ../src/generate_two_string.py 20 > test_random.txt
@echo "a 1 a 1" >> test_random.txt
@cat test_random.txt
@echo "Вывод:"
@../src/algo < test_random.txt
@echo "Ввод: длина строки 40"
@python3 ../src/generate_two_string.py 40 > test_random.txt
@echo "a 1 a 1" >> test_random.txt
@cat test_random.txt
@echo "Вывод:"
@../src/algo < test_random.txt
@echo
```

ПРИЛОЖЕНИЕ Б

ТЕСТИРОВАНИЕ

Таблица 1 - тестирование:

Входные данные:	#Тест 1: одна замена text texx a 1 a 1
Результат:	1 MMMR

Входные данные:	Тест 2: одно удаление text s odnoi slishney bukvoi text s odnoi lishney bukvoi a 1 a 1
Результат:	1 MMMMMMMMMMMMMMMMMDMMMMMMMMMMMMMMMM

Входные данные:	Тест 3: одна вставка zer zero a 1 a 1
Результат:	1 MMMI

Входные данные:	Тест 4: одинаковые строки odinakovii stroki odinakovii stroki a 1 a 1
Результат:	0 MMMMMMMMMMMMMMMMMMMM

Входные данные:	Тест 5: случайные строки разного размера hello iosadjnqwllllwdq
-----------------	---

	1 1 1 1
Результат:	15 IIIIIIIRRMIIIR

Входные данные:	Тест 6: дорогая замена а е е 100 а 1
Результат:	2 ID

Входные данные:	Тест 7: дорогое удаление а е е 90 а 100
Результат:	90 R